Fast-Qwen Architecture & Codebase Bug Report

Date: April 20, 2026 **Target:** `fast-qwen` inference engine **Scope:** Verification of `bugs.pdf` fixes, deep-dive architectural audit against `low-level.pdf` specifications, and Python toolchain review.

Executive Summary

While several historical issues documented in `bugs.pdf` have been successfully patched, the C++, CUDA, and Python codebases remain fundamentally flawed. The engine contains **several CRITICAL (P0) bugs** that will cause compilation failures, hard deadlocks, Numba conversion crashes, or completely corrupted matrix math. Furthermore, it blatantly violates several core architectural tenets laid out in `low-level.pdf`, resulting in severe NUMA latency penalties and cache destruction.

Part 1: Outstanding / Unpatched Legacy Bugs (From `bugs.pdf`)

1. The Taylor Expansion Lie (Empty Stub)

- **Location:** `ssm_kernels.cu`
- **Criticality: CRITICAL (P0)**
- **Description:** The math function `rational_softplus` is defined at the top of the file, but the wrapper `launch_ssm_softplus_taylor` that the orchestrator actually calls is completely empty.
- **Impact:** The Softplus activation step never happens, breaking the SSM recurrence mathematical logic completely. Output generation will be gibberish.

2. 48-Byte Alignment Erasure

- **Location:** `common.h` (`BlockQ4` definition)
- **Criticality: MEDIUM (P2)**
- **Description:** The spec requires fast 128-bit vector loads (`LDG.E.128`). However, `BlockQ4` includes `uint8_t pad[12]`, forcing the struct to exactly 48 bytes.
- **Impact:** 48 bytes is not cleanly divisible by 16 bytes (128-bit). This breaks aligned memory loads, causing the compiler to emit multiple slower, unaligned memory instructions instead of vectorized fetching.

Part 2: Newly Discovered Codebase Failures (C++ & CUDA)

3. Missing `rmsnorm_avx2` Linker Failure

- **Location:** `orchestrator.cpp` (Line ~125)
- **Criticality: CRITICAL (P0)**
- **Description:** The orchestrator calls `rmsnorm_avx2(...)` for the final layer normalization before the LM head. This function signature exists nowhere in `worker_pool.cpp`, `quip_unpack_avx2.cpp`, or `common.h`.
- **Impact:** The application will flat-out refuse to compile, failing with an "unresolved external symbol" error during the linking stage.

4. Catastrophic VRAM Out-of-Bounds in SSM

- **Location:** `ssm_kernels.cu` (`ssm_sram_convolution_kernel`)
- **Criticality: CRITICAL (P0)**
- **Description:** The kernel reads `recurrent_state[2 * 4096 + i]`. As `i` scales up to 4096, it reaches index `12,287`. However, the state allocated per layer is only `4 * HIDDEN_DIM` (4 × 2048 = `8192`) floats.
- **Impact:** Buffer overflow by over 4,000 floats. This will overwrite the subsequent layer's KV history or trigger an immediate CUDA memory segmentation fault (Illegal Memory Access).

5. Attention Head Dimension Truncation

- **Location:** `fused_attention_rotor.cu` (`attention_out_proj_kernel`)
- **Criticality: CRITICAL (P0)**
- **Description:** The upstream kernel outputs `Q_HEADS * HEAD_DIM` (64 × 128 = `8192`) floats. The downstream projection kernel processes a loop bounded by `#define OUT_INNER 4096`.
- **Impact:** The projection kernel completely ignores heads 32 through 63. Half of the attention data is dropped into the void, guaranteeing broken model outputs.

6. Thread ID Mismatch & Worker Deadlock

- **Location:** `orchestrator.cpp` & `worker_pool.cpp`
- **Criticality: CRITICAL (P0)**
- **Description:** The orchestrator pushes MoE calculation jobs to PIDs `1` through `8` using `worker_pool_->submit(k + 1, ...)`. In this architecture, odd PIDs are assigned to "Sibling B" (the DMA prefetcher), which runs an infinite prefetch loop and *never checks* the standard work queue.

- **Impact:** The main thread will call `wait(k + 1)` and instantly deadlock forever, waiting for a signal from a thread that is not executing the work.

7. Undefined Behavior in CUDA Warp Reductions

- **Location:** `fused_attention_rotor.cu`
- **Criticality: CRITICAL (P0)**
- **Description:** The kernel uses `__shfl_xor_sync(0xFFFFFFFF, val, off)` inside a loop where `off` is initialized to `64`.
- **Impact:** A CUDA warp is exactly 32 threads. Using an offset of 32 or greater in a warp shuffle instruction is Undefined Behavior. It will result in corrupted KV scales and fundamentally broken dot-product reductions.

Part 3: Deviations from the `low-level.pdf` Architecture

8. Massive Orchestrator Memory Leaks

- **Location:** `orchestrator.cpp`
- **Criticality: HIGH (P1)**
- **Description:** Gigabytes of memory are allocated in the constructor (`cudaMalloc`, `cudaHostAlloc`). The destructor contains zero calls to `cudaFree` or `cudaFreeHost`.
- **Impact:** Deleting and recreating the orchestrator (e.g., loading a different model or restarting the session) will instantly result in Out-Of-Memory (OOM) failures.

9. False Sharing in the CPU Thread Pool

- **Location:** `common.h` (`SiblingWorkItem` struct)
- **Criticality: HIGH (P1)**
- **Description:** The `SiblingWorkItem` struct adds up to exactly 48 bytes. Because it is smaller than the standard 64-byte `CACHE_LINE`, sequential array items (`g_sibling_works[0]` and `g_sibling_works[1]`) land on the exact same physical cache line.
- **Impact:** As Sibling A polls the `ready` atomic boolean, and Sibling B writes to its own struct, they will trigger non-stop L1/L2 cache invalidations across physical CPU cores (False Sharing). This will cripple CPU performance.

10. The L3 Cache "Tsunami" Violation

- **Location:** `worker_pool.cpp`
- **Criticality: HIGH (P1)**

- **Description:** `low-level.pdf` strictly mandates keeping the lookahead buffer under 16MB to protect the 35MB Broadwell L3 cache. The code instantiates `SPSCRing<WeightTile, 4> tile_ring`. With tiles sized at 16MB, a capacity of 4 allows the prefetcher to hoard 64MB of data.
- **Impact:** Trashes the entire L3 Cache, flushing critical KV headers and intermediate activations to slow DDR4 RAM, resulting in catastrophic latency spikes.

11. Missing Asynchronous MoE Spin-Wait

- **Location:** `orchestrator.cpp`
- **Criticality: HIGH (P1)**
- **Description:** The specification demands zero-copy pinned memory to signal the CPU when the MoE router is done, allowing the CPU to spin-wait for sub-microsecond latency. The code actually calls `CUDA_CHECK(cudaStreamSynchronize(streams_[STREAM_ROUTER]));`.
- **Impact:** Forces an OS-level thread sleep/wake cycle instead of a spin-wait. This completely negates the performance benefit of the pinned memory mapping and halts the CPU/GPU asynchronous pipeline.

12. NUMA Core Affinization Overflow

- **Location:** `worker_pool.cpp`
- **Criticality: HIGH (P1)**
- **Description:** Threads are pinned to cores using `core = 2 + pid + (isb ? 14 : 0)`. With `pid` reaching 13, the assigned core goes up to `29`.
- **Impact:** Broadwell Node-0 only handles cores 0-13 (and HT 14-27). Pushing threads to cores cores 28 and 29 spills the workload onto the second physical CPU socket. This incurs a massive QPI (QuickPath Interconnect) latency penalty, violating the single-socket NUMA isolation strategy.

13. Cache Bypass Contradiction

- **Location:** `worker_pool.cpp`
- **Criticality: MEDIUM (P2)**
- **Description:** The spec dictates bypassing the cache hierarchy using streaming loads (`_mm256_stream_load_si256`). However, the Sibling B prefetcher explicitly fetches data into the cache (`_mm_prefetch(..., _MM_HINT_NTA)`).
- **Impact:** Actively prefetching data into the cache just for the worker thread to bypass the cache is computationally contradictory. It wastes memory bandwidth and instruction cycles

for zero benefit.

Part 4: Python Script & Toolchain Failures

14. The AVX2 Compiler Erasure

- **Location:** `compile.py`
- **Criticality: HIGH (P1)**
- **Description:** In `CL_FLAGS`, standard MSVC arguments are passed, but the vectorization architecture flag (`/arch:AVX2`) is completely omitted despite targeting an AVX2 codebase (`quip_unpack_avx2.cpp`).
- **Impact:** Because MSVC defaults to SSE2 compatibility, failing to pass `/arch:AVX2` will cause the compiler to either throw errors on `_mm256_*` intrinsics or generate extremely slow, unoptimized fallback code, crippling the CPU worker pool.

15. Numba Array View Crash

- **Location:** `convert_gguf.py` (`pack_q4` function)
- **Criticality: CRITICAL (P0)**
- **Description:** Inside the `@njit` decorated function, the script attempts to extract bytes from a float using `.view()`: `np.array([scale], dtype=np.float32).view(np.uint8)`.
- **Impact:** Numba's `nopython` mode strictly prohibits calling `.view()` on dynamically allocated arrays created inside a JIT loop. The conversion script will instantly crash with a `TypingError` during the Q4 packing stage.

16. Subprocess Pipe Deadlock

- **Location:** `test_prompt.py`
- **Criticality: CRITICAL (P0)**
- **Description:** A 60-second timeout check (`if time.time() - start_time > 60: break`) is placed *after* a blocking `char = proc.stdout.read(1)` call.
- **Impact:** Because `read(1)` is blocking, if the C++ engine deadlocks (which is guaranteed due to Bug #6), the read call waits forever. The timeout is never reached, causing the test script to hang permanently.

17. Missing Instruct Chat Template

- **Location:** `chat.py` & `test_prompt.py`
- **Criticality: HIGH (P1)**

- **Description:** User input is tokenized directly (`tokenizer.encode(user_input)`) without applying the required ChatML/Qwen chat template formatting (e.g., `<|im_start|>user\n...<|im_end|>`).
- **Impact:** Qwen models are heavily instruct-tuned. Passing raw text without structural tokens causes the model to perform raw text continuation instead of acting as a chatbot, resulting in confused or hallucinated responses.

18. `NoneType` **Crash on Missing Tensors**

- **Location:** `convert_gguf.py`
- **Criticality: MEDIUM (P2)**
- **Description:** The `fetch_tensor(f"blk.{l}.{s}")` function returns `None` if a tensor isn't found, but the script blindly calls `w.reshape(...)` or `pack_q4(w)` without checking.
- **Impact:** If the GGUF file naming conventions deviate slightly (e.g., `ffn_down.weight` instead of `ffn_down_shexp`), the script will instantly crash with an `AttributeError: 'NoneType' object has no attribute 'reshape'`